

Tech Hiring Assignment

Role: Full-Stack Engineer (Fresher) | Time Budget: 4–6 hours | Submission: Single document only

LOKARANJAN BS

Email : lokaranjan31@gmail.com

Mobile : 7449166481

PROBLEM STATEMENT

Hero Cycles is one of the largest bicycle manufacturers in India. You are a fresher engineer joining their small digital team. On Day 1 your manager gives you this brief:

“We sell thousands of cycle configurations — different frames, different gear sets, different tyre types. Our sales team is constantly going back and forth on pricing because part costs change every few months — a tyre priced at Rs. 200 in January may be Rs. 230 by December. Right now everything is done on Excel sheets. We need a proper pricing engine — something salesperson(s) can use to manage cycle configurations & parts, and instantly get the total price, broken down by component. Can you build this?”

BREAKING DOWN THE PROBLEM

The problem is broken down into teams/domains based on the responsibility. Each domain owns a distinct kind of knowledge that no other domain has:

- **Pricing domain:** This domain handles pricing related tasks are handled. Parts price will be decided here and new parts prices will be updated here and their effective dates.(can be a single person given a small digital team).
- **Config domain:** This domain handles configuration related tasks. The bike base configurations(bicycle configurations) are decided here based on use cases like (Mountain bike, street bike, sport bike) and variants will be configured (regular tyre - steel frame variant, wide tyre - carbon fiber frame variant).
- **Sales domain:** This domain handles sales related tasks. Customer interaction, handling customer requests, addressing the configuration request and conveying the pricing details to customer.

1. Questions About the Problem Statement

Pricing Domain

- Will the price of the parts change multiple times a day?
- Should the system remember the reason for the price change ?
- Should the system remember the past price history?
- Will the prices get updated in bulk or one by one?
- Will the pricing team explicitly publish the price in specific time of a day?
- Is pricing team a single person or multiple?
- Should the availability of parts be tracked?
- If price of a bike variant in the morning until 12pm is Rs.3500 and is updated to Rs.3800 after 12pm due to change in parts price. If a customer purchases before 12pm and another customer purchases after 12pm. Will the each customer pay different price for the same configuration?
- Will the sales team get notified if the price changes?
- Will the prices of the variants gets stored in the DB?

Config Domain

- Who decides the compatibility between parts?
- Will the config team configure all parts of a cycle configuration?
- Will there be dependency among parts, if exists who handles that?
- Are add-on's like bottle holder, head light etc provided? If provided will have dependency or compatibility rules and who decides it?
- How does the Config team handle the add-on parts?
- What happens if the config domain does a mistake and enters a duplicate entry?
- What happens if the config team does not mention all the parts that a bicycle requires?

Pricing And Config domain

- How are add-on parts being handled by the pricing team?
- Are the add-on parts billed separately ?

Sales Domain

- Does the sales person freely modify/swap individual parts?
- Is a bill or a quotation saved when a customer doesn't purchase today, or recomputed fresh whenever they return?
- Who decides whether a given add-on is compatible to chosen config ?

- Should sales team or the customer see part-level pricing or only the total?
- Is price breakdown provided for add-on's?
- Is there a option for discount in the total price and is tax included in the final price?

2. Assumptions Made

2.1 Pricing Domain

- The price of the parts will not change multiple times in a day.
- The system will not remember the past price history.
- The reason for a price change (transport cost, demand, etc.) is not stored in the system. It is just an assumption for why prices change, not something the system needs to remember.
- The prices will get updated in bulk.
- The pricing team will publish the price and the effective date from which the price is valid or applied.
- Two customers purchasing the same cycle configuration on different times of the same day will not get different prices. When pricing team publishes a new price, it is tagged with an effective date (always the next day). The system checks this effective date whenever the price is requested, and only applies the new price once that date has arrived. So every request within the same day gets the same price, no matter how many people ask or when
- A published price update takes effect starting the next day, not the same day it is published.
- Same parts used in two different configurations will have the same partId and same price.
- Pricing team publishes updates manually no schedulers are used.
- The parts availability is not tracked.
- There is no notification sent to the pricing team about price change. The sales person will simply see the current price whenever they start a new request.
- The variant prices will get stored in the table.

2.2 Config Domain

- The config team decides the compatibility between the parts. For example, a 4in wide tire should have a compatible 4in wide rim.
- Yes the config team decides the compatibility between all the mandatory parts and as well as add-on parts. Config team defines complete, pre-approved Variants per base configuration— each variant is a full, already-validated bundle of parts.
- The config team resolves the dependency between parts. For example, a mountain bike should have a wider tire for more grip and A four-inch wide tire should have a four-inch wide compatible rim, now as the rim size is increased compatible breaks should also be added. This applies not only to tyres, also for all the other mandatory parts.(frames, rims, tyres, breaks, disc, handle bar etc..)
- Add-ons: config team defines which add-ons are valid per base configuration; salesperson or customer can freely choose among valid add-ons with no per-pick approval needed.
- If the config team enters a duplicate entry TYRE→ 1 and TYRE→ 1, a validation will take place which does not allow duplicates.
- All the parts in the table part_category are considered, if the final list misses on any one the part. An exception will be thrown.

2.3 Pricing And Config domain

- The add-on parts are handled in the same way the pricing team handles the mandatory parts. The pricing team will publish a new price, and the new price will be effective from the next day.

2.4 Sales Domain

- Sales person cannot swap between the individual parts. He can only select between the pre configured parts.
- The quotation of a customer not making purchase will not be saved. It will be recomputed the next time they return.
- The compatibility of add-on's is decided by the config team.
- The customer or the sales person will not see the detailed price breakdown between parts. they can only see the total price. As this will reveal the business model to the competitors and also causes unwanted confusion between sales person and the customer.

- Add-on's prices are shown individually part wise. As they can be added or removed based on customer request.
- Total price = variant price (bundle) + sum of selected add-on prices. No tax, no discount included.

FLOW OF THE PROBLEM STATEMENT

1. When parts come in the prices of the parts will be decided based on certain parameters like transport cost, handling charges etc... (The parts pricing section will be managed by the pricing team, assumed to be a single person given the small team).
2. The cycle configuration will have a set of use cases, which are considered as base configurations (Eg: mountain use, street use, sport use cycles etc...). From the base configurations the cycles are divided into variants which will have different frames, different tyre types, different gear sets as mentioned in the question. (The variants will be configured by the Configuration team, ensuring regulations and customer safety. As an option of freely selecting parts can lead to compatibility issues and safety concerns).
3. When a customer requests for a bike configuration, for example "Street cycle with wide tyres and carbon fiber frame". The sales person will first select the bike configuration (Street cycle) then he will choose the variants requested by the customer (wide tyre carbon fiber frame variant). The sales person will be able to see the price of the variant requested by the customer. (The sales section is handled by the Sales team)

3. Solution Approach

3.1 Class Model

Part

Stores the details of an individual part along with its current price, new price and its effective date. New price will only be effective from the effective date. Each Part belongs to one PartCategoryRule, which defines its default quantity in a variant.

AddOn

Stores optional accessories that can be attached to a bike configuration. Similar to Part, it maintains both the new price and its effective date.

PartCategoryRule

Stores the default quantity of each part category (e.g., Pedal = 2, Frame = 1). During variant price calculation, this quantity is multiplied with the current price of the individual selected part.

Variant

Represents a bike variant under a Bike Configuration. One variant will have Stores the pre-calculated currentPrice, which will be calculated again only when scheduled part prices become effective.

BikeConfiguration

Represents the base bike model. A Bike Configuration can contain multiple Variants and multiple valid AddOns.

PricingService (all pricing computation):

addParts(requestDto)

Creates a new Part after validating that the Part name is unique. Associates the Part with its corresponding Part Category Rule and stores its initial current price.

calculateVariantPrice(variantId)

- Calculates the total Variant price by retrieving all Parts associated with the Variant through the VariantPart junction table. For each Part, multiplies its current price with the default quantity based on category from PartCategoryRule and returns the total.

getVariants(bikeConfigId)

- Returns all Variants belonging to a Bike Configuration together with their cached prices for display to the Sales Team.

getValidAddOnsWithPrices(bikeConfigId)

- Returns all valid AddOns associated with a Bike Configuration together with their current prices.

calculateAddonsPrices(addonIds)

- Calculates the combined price of the selected AddOns using their current prices and returns both the selected AddOns and their total price.

calculateTotalPrice(variantId, addonIds)

- Calculates the final quotation by combining the stored Variant price with the total price of the selected AddOns and returns the total price along with the list of selected AddOns.

updatePartPrices(partPrices, effectiveFrom)

- Allows the Pricing Team to schedule future Part prices in bulk. Stores the future prices (new price) and effective date without modifying the current prices. Has a partial success handling mechanism , which reports the invalid entries while processing valid updates, returns the successful updates and failed updates.

updateAddOnPrices(addOnPrices, effectiveFrom)

- Allows the Pricing Team to schedule future AddOn prices in bulk using the same partial-success approach as Part pricing.

updateEffectivePrices() (for pricing scheduler)

- Executed by the scheduler every day. Updates scheduled Part prices by promoting newPrice to currentPrice and clearing the new price and effective date fields.
- Identifies the variants affected by the new Part price updates and recalculates the affected variant's price and updates stored value.
- Executed by the scheduler every day. Activates scheduled AddOn prices by promoting newPrice to currentPrice and clearing the new price and effective date fields.

updateEffectiveAddOnPrices() (for pricing scheduler)

ConfigService (all configuration authoring)

createBikeConfig(name)

- Creates a new Bike Configuration that serves as the base model for one or more Variants.

validateVariantParts(partDtos)

- Validates the selected Parts before creating a Variant by ensuring No duplicate Part Categories are selected and every Part Category defined in PartCategoryRule is selected exactly once. This guarantees that every Variant contains exactly one Part from each Part Category.

createVariant(name, bikeConfigId, partDtos)

- Creates a Variant under the selected Bike Configuration and stores all corresponding VariantPart details within a single transaction. After successful validation and creation, calculates the Variant price using pricingService.calculateVariantPrices(variantId) and stores it as the Variant's current price.
-

PriceScheduler

updateEffectivePrices()

Runs automatically every day at **12:00 AM**. Invokes:

- PricingService.updateEffectivePrices()
- PricingService.updateEffectiveAddOnPrices()

to activate all scheduled Part and AddOn prices and refresh the cached prices of affected Variants. **Deliberately excluded from the class model**

- Pricing Team and Config Team as entities - no independent data or relationships to store, as history, past prices are not included. They act like roles, not entities with data..
- VariantPart and ConfigurationAddOn are classes, they are pure junction tables with no independent behaviour, needed only to satisfy normalization for the many-to-many relationships.
- A separate SalesService is not created. The sales flow only calls existing methods from BikeConfig and PricingService one after another. Since it does not do any work of its own, adding a new class for it would only add extra complexity without any real benefit.
- Role based access control is assumed to exist and not implemented given the time constraint. (ROLE_PRICING, ROLE_CONFIG, ROLE_SALES) can be implemented.

3.2 Data Model

bike_configurations

Column	Type	Notes
id	INT (PK)	
name	VARCHAR	Mountain, Street, Sport, etc.

variants

Column	Type	Notes
id	INT (PK)	
name	VARCHAR	e.g. Standard, Wide tyre
current_price	FLOAT	Calculated price of variant
status	ENUM	VALID / INVALID
bike_configurations_id	INT (FK)	bike_configurations.id

parts

Column	Type	Notes
id	INT (PK)	
name	VARCHAR	Steel front wheel, steel back wheel, carbon frame, etc...
current_price	FLOAT	current active price
new_price	FLOAT, nullable	New future price
effective_from	DATE, nullable	New price effective date
part_category_rule_id	INT(FK)	part_category_rule_id

parts_category

Column	Type	Notes
id	INT (PK)	
category	VARCHAR	Name of the Part Category (FRAME, TYRE, etc.).
default_quantity	INT	Default quantity of a category (Eg: TYRE =2, WHEEL=2, FRAME=1)

variant_parts (junction — resolves Part ↔ Variant many-to-many)

Column	Type	Notes
id	INT (PK)	
parts_id	INT (FK)	parts.id
variants_id	INT (FK)	variants.id

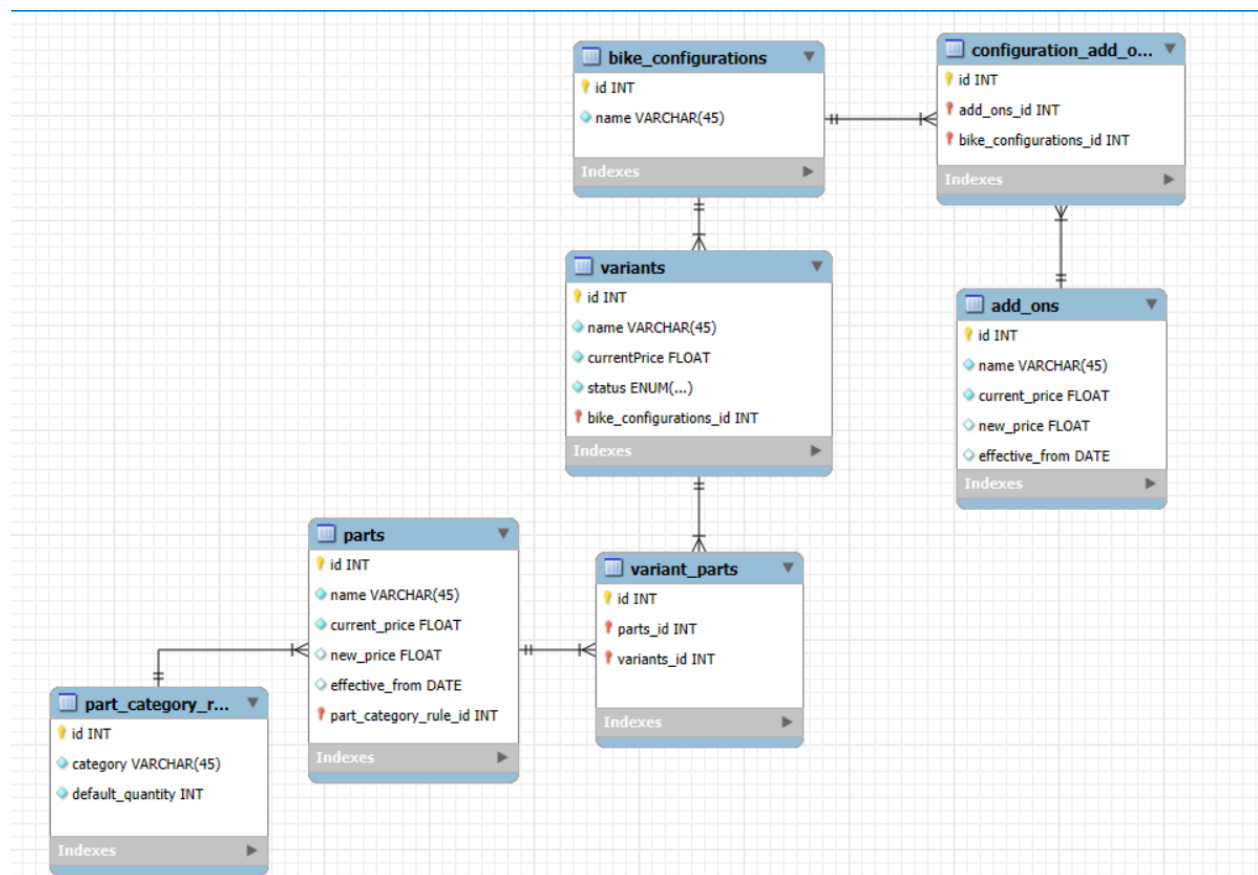
add-ons

Column	Type	Notes
id	INT (PK)	
name	VARCHAR	
current_price	FLOAT	same pricing mechanism as parts
new_price	FLOAT, nullable	New future price
effective_from	DATE, nullable	New price effective date

configuration_add_ons (junction — resolves AddOn ↔ BikeConfig many-to-many)

Column	Type	Notes
id	INT (PK)	
bike_configurations_id	INT (FK)	bike_configurations.id
add_ons_id	INT (FK)	add-ons.id

Data Model Diagram:



3.3 Service Model & Pseudocode

Read methods - Sales Person browsing — PricingService methods

Getting Variants with Prices:

Response:

```
VariantPriceDto {  
    String variantName;  
    Float variantPrice;  
}
```

```
public List<VariantPriceDto> getVariants(Long bikeConfigId){  
    List<VariantPriceDto> result = new ArrayList<>();  
    List<Variant> variants= variantRepository.findByConfigurationId(bikeConfigId);  
    for(Variant variant : variants){  
        VariantPriceDto dto=new VariantPriceDto();  
        dto.setVariantName(variant.getName());  
        dto.setVariantPrice(variant.getCurrentPrice());  
        result.add(dto);  
    }  
    return result;  
}
```

Getting addOn with prices:

Response:

```
AddonPriceDto {  
    String addonName;  
    float addonPrice;  
}
```

```
public List<AddonsWithPriceResponseDto> getValidAddOnsWithPrices(Long  
bikeConfigId){  
    List<AddonsWithPriceResponseDto> result = new ArrayList<>();  
    List<Long> addOnIds=  
configurationAddOnRepository.findByConfigurationId(bikeConfigId);  
    List<AddOn> addOns = addOnRepository.findByIdIn(addOnIds);
```

```

for(AddOn addOn : addOns){
    AddonsWithPriceResponseDto dto = new AddonsWithPriceResponseDto();
    dto.setAddonName(addOn.getName());
    dto.setAddonPrice(addOn.getCurrentPrice());
    result.add(dto);
}
return result;
}

```

Price calculation :

=> Calculate variant price

```

public Float calculateVariantPrice(Long variantId){
    List<VariantPart> variantParts= variantPartRepository.findByVariantId(variantId);

    float total = 0f;
    for(VariantPart varPart : variantParts){

        Part part = partRepository.findById(varPart.getPartId()).orElseThrow(
            ()->new PartNotFoundException("Part not found")
        );

        total += part.getCurrentPrice() * part.getPartCategoryRule().getDefaultQuantity();
    }
    return total;
}

```

Calculate addOn prices and return addOn individual prices with addOn names

Response:

```
AddonResponseDto {  
    List<AddOn> addons;  
    Float totalPrice;  
}
```

```
public AddonResponseDto calculateAddonsPrices(List<Long> addonIds ){  
    float total = 0f;  
    List<AddOn> addons = addOnRepository.findByIds(addonIds);  
    AddonResponseDto response = new AddonResponseDto();  
    for(AddOn addon :addons){  
        total = total + addon.getCurrentPrice();  
    }  
    response.setAddons(addons);  
    response.setTotalPrice(total);  
    return response;  
}
```

Final Price calculation and AddOn price breakdown

Response:

```
TotalPriceDto {  
    float variantPrice;  
    AddonResponseDto addonResponse;  
    float finalTotal;  
}
```

```
public TotalPriceDto calculateTotalPrice(CalculateTotalPriceRequestDto requestDto){  
  
    Variant variant= variantRepository.findById(requestDto.getVariantId()).orElseThrow(  
        ()-> new VariantNotFoundException("Variant not found")  
    );  
    AddonResponseDto addonResponse = calculateAddonsPrices(requestDto.getAddonIds());  
  
    float finalTotal = variant.getCurrentPrice() + addonResponse.getTotalPrice();  
  
    return new TotalPriceDto(variant.getCurrentPrice(), addonResponse, finalTotal);}
```

Price update — (accessed by pricing team's) - partial-success feature

```
FailureDto {  
    Long partId;  
    String reason;  
}
```

Response:

```
BulkUpdateResponseDto {  
    List<Long> successIds ;  
    List<FailureDto> failures;  
}
```

Request:

```
BulkUpdatePartRequestDto {  
    private HashMap<Long,Float> partPrices;  
    private LocalDate effectiveFrom;  
}
```

```
public BulkUpdateResponseDto updatePartPrices(BulkUpdatePartRequestDto requestDto){  
    List<Long> partIds=new ArrayList<>(requestDto.getPartPrices().keySet());  
    List<Part> parts = partRepository.findByIdIn(partIds);  
  
    List<Long> successIds = new ArrayList<>();  
    List<FailureDto> failures = new ArrayList<>();  
  
    for(Part part: parts){  
        try {  
            part.setNewPrice(requestDto.getPartPrices().get(part.getId()));  
            part.setEffectiveFrom(requestDto.getEffectiveFrom());  
            successIds.add(part.getId());  
  
        }catch (InvalidDataEnteredException ex){  
            failures.add(new FailureDto(part.getId(),ex.getMessage()));  
        }  
    }  
    return new BulkUpdateResponseDto(successIds,failures);  
}
```

Addon price update — (accessed by pricing team) - partial-success-feature

Request:

```
BulkUpdatePartRequestDto {  
    private HashMap<Long,Float> partPrices;  
    private LocalDate effectiveFrom;  
}
```

Response:

```
BulkUpdateResponseDto {  
    List<Long> successIds ;  
    List<FailureDto> failures;  
}
```

```
public BulkUpdateResponseDto updateAddOnPrices(BulkUpdateAddOnRequestDto  
                                              requestDto){  
    List<Long> addOnIds=new ArrayList<>(requestDto.getAddOnPrices().keySet());  
    List<AddOn> addOns = addOnRepository.findByIdIn(addOnIds);  
  
    List<Long> successIds = new ArrayList<>();  
    List<FailureDto> failures = new ArrayList<>();  
  
    for(AddOn addOn: addOns){  
        try {  
            addOn.setNewPrice(requestDto.getAddOnPrices().get(addOn.getId()));  
            addOn.setEffectiveFrom(requestDto.getEffectiveFrom());  
            successIds.add(addOn.getId());  
  
        }catch (InvalidDataEnteredException ex){  
            failures.add(new FailureDto(addOn.getId(),ex.getMessage()));  
        }  
    }  
    return new BulkUpdateResponseDto(successIds,failures);  
}
```

Scheduler methods: in Pricing Service

Update effective part prices:

```
public void updateEffectivePrices() {
    List<Part> parts=partRepository
        .findByNewPricesNotNullAndEffectiveFromLessThanEqual(LocalDate.now());
    List<Long> partIds=new ArrayList<>();
    for(Part part:parts){
        part.setCurrentPrice(part.getNewPrice());
        part.setNewPrice(null);
        part.setEffectiveFrom(null);
        partIds.add(part.getId());
    }
    List<Long> variantIds = variantPartRepository
        .findDistinctVariantIdsByPartIdIn(partIds);

    for(Long variantId:variantIds){
        Variant variant = variantRepository.findById(variantId)
            .orElseThrow(() -> new RuntimeException("Variant not found"));

        variant.setCurrentPrice(calculateVariantPrice(variantId));
    }
}
```

Update effective AddOn prices:

```
public void updateEffectiveAddOnPrices() {
    List<AddOn> addOns = addOnRepository
        .findByNewPricesNotNullAndEffectiveFromLessThanEqual(LocalDate.now());
    for (AddOn addOn : addOns) {
        addOn.setCurrentPrice(addOn.getNewPrice());
        addOn.setNewPrice(null);
        addOn.setEffectiveFrom(null);
    }
}
```

Scheduler:

```
@Scheduled(cron = "0 0 0 * * *")
public void updateEffectivePrices() {
    pricingService.updateEffectivePrices();
    pricingService.updateEffectiveAddOnPrices();
}
```


Add Part - (accessed pricing team):

Request:

```
PartRequestDto {  
    private String partName;  
    private Long partCategoryId;  
    private Float currentPrice;  
}
```

```
public String addParts(PartRequestDto requestDto){  
    if(partRepository.existsByName(requestDto.getPartName())){  
        throw new DuplicatePartException("Part name already exists");  
    }  
    Part part= new Part();  
    part.setName(requestDto.getPartName());  
    part.setCurrentPrice(requestDto.getCurrentPrice());  
  
    PartCategoryRule category=  
        partCategoryRuleRepository.findById(requestDto.getPartCategoryId()).  
            orElseThrow(  
                ()->new PartCategoryNotFoundException("Category not found")  
            );  
  
    part.setPartCategoryRule(category);  
    partRepository.save(part);  
    return "Part added successfully";  
}
```

ConfigService - Config Domain methods

Create BikeConfig

```
BikeConfigurationResponseDto createBikeConfig(String name){  
    BikeConfiguration bikeConfig = new BikeConfiguration();  
    bikeConfig.setName(name);  
    bikeConfigurationRepository.save(bikeConfig);  
    BikeConfigurationResponseDto dto=new BikeConfigurationResponseDto();  
    dto.setName(name);  
    return dto;  
}
```

Create Variant

Request:

```
VariantPartRequestDto {  
    private Long partId;  
    private Integer quantity;  
}
```

```
public VariantResponseDto createVariant(String name, Long bikeConfigId,  
                                       List<VariantPartRequestDto> partDtos) {  
    //validates the parts  
    validateVariantParts(partDtos);  
  
    BikeConfiguration bikeConfiguration = bikeConfigurationRepository  
        .findById(bikeConfigId).orElseThrow(() ->  
            new RuntimeException("Bike Configuration not found"));  
  
    Variant variant = new Variant();  
    variant.setName(name);  
    variant.setBikeConfiguration(bikeConfiguration);  
    variant.setStatus(VariantStatus.ACTIVE);  
  
    variant = variantRepository.save(variant);  
  
    float totalPrice = 0f;  
  
    for (VariantPartRequestDto dto : partDtos) {  
        Part part = partRepository.findById(dto.getPartId())  
            .orElseThrow(() ->  
                new PartNotFoundException("Part not found"));  
  
        VariantPart variantPart = new VariantPart();  
        variantPart.setVariantId(variant.getId());  
        variantPart.setPartId(part.getId());  
        variantPartRepository.save(variantPart);  
        totalPrice=pricingService.calculateVariantPrice(variant.getId());  
    }  
    variant.setCurrentPrice(totalPrice);  
    variantRepository.save(variant);  
  
    return new VariantResponseDto(variant.getId(), variant.getName(),  
        variant.getCurrentPrice()  
    );}
```

Validate Variant parts:

```
private void validateVariantParts(List<VariantPartRequestDto> partDtos) {

    Set<Long> selectedCategoryIds = new HashSet<>();

    for (VariantPartRequestDto dto : partDtos) {

        Part part = partRepository.findById(dto.getPartId())
            .orElseThrow(() ->
                new PartNotFoundException("Part not found"));

        Long categoryId = part.getPartCategoryRule().getCategoryId();

        if (!selectedCategoryIds.add(categoryId)) {
            throw new DuplicatePartCategoryException(
                "Only one part can be selected for category: "
                + part.getPartCategoryRule().getCategory());
        }
    }

    List<PartCategoryRule> allCategories = partCategoryRuleRepository.findAll();

    if (selectedCategoryIds.size() != allCategories.size()) {
        throw new InvalidDataEnteredException(
            "Exactly one part must be selected for every part category.");
    }
}
```

Create AddOn

```
AddOn createAddOn(String name, float price){
    AddOn addOn = new AddOn();
    addOn.setName(name);
    addOn.setPrice(price);
    AddOnRepo.save(addOn);
    return addOn;
}
```

Link AddOn to BikeConfig

```
void linkAddOnToConfig(Long addOnId, Long bikeConfigId){
    ConfigurationAddOn link = new ConfigurationAddOn();
    link.setAddOnId(addOnId);
    link.setBikeConfigId(bikeConfigId);
    ConfigurationAddOnRepo.save(link);
}
```

4. UI and Backend Coding Approach

4.1 Device

- The primary device considered here is a mobile or a tablet. As it is carried by a sales person, and is easy to access price in presence of customer.
- The same screen flow and logic can be implemented for a desktop layout, not designed separately here, since the flow and backend calls stay identical.

4.2 Sales Flow - Screen by Screen

Screen 1 - Bike Config Selection

- Will contain the list of bike configurations. The sales person has to select a configuration among the available options.
- Tap next ---- proceeds to Screen 2.

Screen 2 — Variant Selection (with prices)

- Screen 2 will contain List of variants with prices based on the bike config selected. Each row shows variant name and the price of the variant. Making it easy for the sales person and customer to view and compare prices.
(e.g., Standard - ₹5000,
Wide Tyre - ₹5200).
- The sales person has to select the variant based on customer request.
- Tap one variant → proceeds to Screen 3. Back button returns to Screen 1.

Screen 3 — Add-on Selection (optional, multi-select)

- Screen 3 contains list of addOns for the selected bike config. Each row showing the addOn name along with the price. This will be a checkbox as a customer can add multiple addOns.
- If customer does not need any addOns sales person can proceed with zero add-ons selected.
- Back button returns to Screen 2 (variant selection can be changed; add-on selections reset or are re-evaluated against the newly selected config's valid list).

Screen 4 — Total / Summary

- Shows: selected variant name + its addOns total price (variant parts not broken down — business decision, see Section 2), each addOn selected will be listed individually with its own price, and the grand total.
- Back button returns to Screen 3 to adjust add-ons, or further back to Screen 2 to change variant.
- Every time the flow moves forward again after a back-navigation, the total is recomputed fresh from whatever selection is currently active — consistent with the stateless, no-persisted-quote design; there is no risk of showing a stale total after changing a selection.

4.3 Config Team Screen

- Create Bike Config - a name field and submit button, calling `createBikeConfig(name)`.
- Create Variant - select an existing bike config, name the variant, choose parts from a checklist, submit as one action - calling `createVariant(name, bikeConfigId, partIds)` atomically, so no variant is ever saved incomplete.
- Add-ons - a form to create a new add-on (name + price, calling `createAddOn`).
- A checklist to link an existing add-on to a config's valid list (calling `linkAddOnToConfig`) - kept as a separate action from variant creation since add-ons are introduced independently over time.

4.4 Pricing Team Screen

- A table showing all parts with their current price, and an editable 'newPrice' + 'effectiveFrom' field per row.
- A single 'Publish' action sends all edited rows together, calling the bulk updatePrices() method, results are shown back per part, consistent with the partial-success handling already built into that method.
- Excel/CSV upload can be a future development, where the team updates the prices in an Excel sheet. Then submitting, will automatically publish the new prices.

5. Prompts Used

Understanding the problem / breaking it down

- Just help me understand the problem. First I will tell the flow, domains, entities. You just tell me how good the system is.
- Shared my own description of the Pricing, Config, and Sales domain flows and asked for review on where improvement was needed (done multiple times as I revised each domain).
- Based on the question asked, what are the remaining gaps in my design?
- How good is my step 1?

Deciding on pricing behaviour

- Is there any other way for price update?
- What if the pricing team updates the price today and it has to be effective from next day morning 6?
- Proposed several versions of the price-update mechanism myself (a next_price-only approach, a scheduler-based approach, a lazy on-demand check) and asked which was better, and why.
- Is there a requirement for me to keep track of price history?
- How to solve discounting problem?

Class model / data model / service model concepts

- Does class model mean deciding on entities?
- I can't understand what each one is. Can you give me an example?
- Can you explain what is class model, data model, and the other one in this scenario?
- Proposed my own table structures (including an incorrect fixed-column design) and asked why it wouldn't work; asked "so it does not qualify normalization?"

Reviewing my own pseudocode

- Submitted my own draft methods (e.g., getVariants, calculateVariantPrice, calculateAddonsPrices, calculateTotalPrice, setNewPrice, updatePrices, createVariant) and asked for error correction, multiple rounds per method.
- Correct the errors in this method.
- Instead of calling this method once for every variant id, wouldn't it be better if I sent a list of variant ids?

Deciding scope / what to include

- Should pricing team require an entity?
- Is there anything left out in the class model, according to the question?
- Should it be a separate service, as sales service?

UI / Part 3 design

- Now shall we decide on Part 3?
- Let's make the device carried by sales person a mobile or a tablet. Should I do it for laptop also?
- Proposed my own screen flow (config → variant → add-ons → total) and asked for review.
- Proposed my flow UI design about Pricing Domain and Config Domain.

Document structure and assignment compliance

- How well is my proposed solution aligned with the requirement asked?